

RECORD STUDIO

WhiteCollar Assessments

Technical Documentation

API Keys · Assessment Creation · Candidate Assignment

Version 1.0.0

Date May 2026

Product Record Studio — WhiteCollar Module

Classification Internal / Developer

Platform Overview

Record Studio WhiteCollar is a skill-verification SaaS platform designed for enterprise hiring teams. The WhiteCollar module provides end-to-end assessment capabilities for white-collar roles — from creating tailored question banks to proctored candidate evaluation and result analytics.

Operating Modes

All resources in the platform are scoped to one of two modes:

Mode	Description	Use Case
test	Sandbox environment — no real candidate impact	Development, QA, integration testing
live	Production environment — real candidate evaluations	Active hiring campaigns

Important

Always create test-mode API keys and assessments during integration development.

Switching from test to live mode filters all data independently — records do not cross modes.

Module Capabilities

- AI-powered assessment generation from job descriptions
- Customized manual question paper builder (Knowledge + Skill sections)
- Batch candidate assignment with duplicate detection
- Real-time proctoring and violation tracking
- Automated scoring and pass/fail determination
- Overview dashboard with funnel analytics

Role-Based Access Control

Every API operation is gated by the requester's organization role:

Role	Create	Update	Delete	Read
owner	Yes	Yes	Yes	Yes
superadmin	Yes	Yes	Yes	Yes
reader	No	No	No	Yes

Error Codes

HTTP Code	Meaning
200	Success
201	Created
400	Validation error — check request body
401	Unauthorized — invalid or missing access token
403	Forbidden — insufficient role for this operation
404	Resource not found
500	Internal server error

01 API Keys

API Keys authenticate external services and integrations with the Record Studio API. Every key is mode-scoped — a test key cannot be used against live endpoints and vice versa.

1.1 Key Properties

Field	Type	Description
keyId	string	Unique identifier (e.g. key_test_abc123)
apiKey	string	Public API key sent in x-api-key header (ak_test_...)
secretKey	string	Secret key — shown ONCE at creation, never retrievable
mode	"test" "live"	Environment scope of the key
name	string	Human-readable label
orgId	string	Owning organisation identifier
createdAt	ISO 8601 date	Timestamp of key creation

WARNING — Secret Key

The secretKey is hashed server-side immediately after creation.

It is returned exactly once in the creation response.

If lost, you must delete the key and create a new one.

Always store it in a secure environment variable — never in source code.

1.2 Endpoints

List API Keys

Returns all keys for the organisation (secretKey excluded from all responses).

```
GET /api/v1/api-keys?orgId={orgId}&mode={mode}
Authorization: Bearer {accessToken}
```

Create API Key

```
POST /api/v1/api-keys?orgId={orgId}
Authorization: Bearer {accessToken}
Content-Type: application/json

{
  "name": "Production Integration",
  "mode": "test" | "live",
  "projectId": "proj_xxx" // optional
}
```

Response — key created (secretKey included one time only):

```
{
  "success": true,
  "data": {
    "keyId": "key_test_4f3a9c...",
    "apiKey": "ak_test_8e2b...",
    "secretKey": "sk_test_f1c9e8..." , // SHOWN ONCE
    "mode": "test",
    "name": "Production Integration",
    "createdAt": "2026-05-30T10:00:00.000Z"
  }
}
```

Delete API Key

```
DELETE /api/v1/api-keys/{keyId}?orgId={orgId}
```

```
Authorization: Bearer {accessToken}
```

1.3 HMAC-SHA256 Request Signing

Every API request must include a cryptographic signature computed with the `secretKey`. This prevents replay attacks and ensures request integrity.

Signature Algorithm

- Concatenate the timestamp (Unix ms) and the JSON-serialised request body.
- Compute HMAC-SHA256 of that string using the `secretKey`.
- Send all three values in request headers.

Headers Required

Header	Value
x-api-key	Your public apiKey
x-timestamp	Current Unix timestamp in milliseconds (string)
x-signature	Hex-encoded HMAC-SHA256 signature

Node.js Example

```
const crypto = require('crypto');

const API_KEY   = process.env.RECORD_STUDIO_API_KEY;
const SECRET    = process.env.RECORD_STUDIO_SECRET_KEY;

const timestamp = Date.now().toString();
const payload   = JSON.stringify(requestBody);

const signature = crypto
  .createHmac('sha256', SECRET)
  .update(timestamp + payload)
  .digest('hex');

// Attach to every request:
headers['x-api-key']   = API_KEY;
```

```
headers['x-timestamp'] = timestamp;  
headers['x-signature'] = signature;
```

Timestamp Window

The server rejects requests where the timestamp differs from server time by more than:

- 5 minutes in Live mode
- 30 minutes in Test mode

Ensure your system clock is synchronised (NTP recommended).

02 Assessment Creation

Assessments are the core evaluation unit in the WhiteCollar platform. An assessment bundles a job context, a scoring configuration, and a set of questions (knowledge sections and/or coding problems) into a publishable candidate experience.

2.1 Assessment Types

Type	Creation Method	Description
AI Powered Assessment	ai	System generates tailored Q&A sets from job title, description, and skills
Customized Assessment	custom	Fully manual — builder UI for knowledge sections and coding problems
Smart AI Assessment	ai	JD/resume hybrid — combines structured data with AI generation

2.2 Role Type & Assessment Scope

Role Type	Knowledge Assessment	Skill Assessment (Coding)	Steps in Builder
IT	Yes	Yes	4 steps
NON-IT	Yes	No (skipped)	3 steps

NON-IT Behaviour

When roleType is set to NON-IT, Step 3 (Skill Assessment) is automatically hidden in the builder UI. Coding problems are excluded from the publish payload.

2.3 Assessment Fields

Field	Type	Description
assessmentId	string	Unique ID (asmnt_xxxx)
name	string	Assessment name (max 150 chars)
assessmentType	string	"AI Powered Assessment" / "Customized Assessment"
creationMethod	"ai" "custom"	How it was created
mode	"test" "live"	Environment scope
roleType	string	"IT" or "NON-IT"
jobTitle	string	Target job title (max 150 chars)
jobDescription	string	Job description (max 1500 chars)
experienceRange	string	e.g. "1-3 years"
skills	string[]	Up to 5 skill identifiers
questionSetType	string	"Same For All" or "Different For All" (AI only)
totalMarks	number	Configured total marks for the assessment
passMarks	number	Minimum marks required to pass
duration	number	Time limit in minutes
difficulty	string	"Easy" / "Medium" / "Hard" / "Expert"
sections	Section[]	Knowledge assessment question sections
codingProblems	Problem[]	Coding challenges (IT roleType only)
isPublished	boolean	Whether the assessment is published to candidates
isActive	boolean	Whether the assessment accepts new candidates

2.4 API Endpoints

Create Assessment

```
POST /api/v1/assessments
Authorization: Bearer {accessToken}
Content-Type: application/json

{
  "orgId": "RORG123456",
  "name": "Frontend Developer Assessment",
```

```

"assessmentType": "Customized Assessment",
"creationMethod": "custom",
"mode":          "live",
"jobTitle":      "Frontend Developer",
"jobDescription": "We are looking for...",
"roleType":      "IT",
"experienceRange": "1-3 years",
"skills":         ["React", "TypeScript"],
"totalMarks":    100,
"passMarks":      65,
"duration":       75,
"difficulty":     "Medium"
}

```

Get Assessment

```

GET /api/v1/assessments/{assessmentId}?orgId={orgId}
Authorization: Bearer {accessToken}

```

List Assessments

```

GET /api/v1/assessments?orgId={orgId}&mode={mode}&page=1&limit=10
Authorization: Bearer {accessToken}

```

Update Assessment / Save Builder Content

A single PUT endpoint handles all updates: metadata, knowledge sections, coding problems, and publishing.

```

PUT /api/v1/assessments/{assessmentId}
Authorization: Bearer {accessToken}
Content-Type: application/json

{

```

```
"orgId": "RORG123456",
"sections": [...], // Knowledge questions (see 2.5)
"codingProblems": [...], // Coding problems (see 2.6)
"isPublished": true // Publish the assessment
}
```

Toggle Active / Delete

```
PATCH /api/v1/assessments/{assessmentId}/toggle?orgId={orgId}
DELETE /api/v1/assessments/{assessmentId}?orgId={orgId}
Authorization: Bearer {accessToken}
```

2.5 Knowledge Section Structure

Section Type	Description	Answer Requirement
single_choice	Multiple choice — one correct answer	Mark exactly one answer isCorrect: true
true_false	Binary choice — True or False	Mark the correct option isCorrect: true
short_answer	Free-text — AI-graded against model answer	Provide model answer text in answers[0]
long_answer	Extended free-text — AI-graded against criteria	Provide model answer text in answers[0]

Section JSON payload example:

```
{
  "sectionId": "sec_abc",
  "sectionType": "single_choice",
  "pointsPerQuestion": 5,
  "questions": [
    {
      "questionId": "q_001",
```

```

    "questionText": "Which hook manages side effects in React?",
    "marks":      5,
    "answers": [
      { "answerId": "a_1", "answerText": "useEffect", "isCorrect": true },
      { "answerId": "a_2", "answerText": "useState",  "isCorrect": false },
      { "answerId": "a_3", "answerText": "useMemo",   "isCorrect": false }
    ]
  }
]
}

```

Marks Validation

Before publishing, the system validates that the sum of all question marks equals the assessment's configured totalMarks.

If they differ, the Finish / Publish button is disabled and a mismatch banner is shown.

2.6 Coding Problem Structure (IT Roles Only)

Field	Type	Description
problemId	string	Unique problem identifier
title	string	Problem title (max 300 chars)
marks	number	Marks awarded for this problem
allowedLanguages	string[]	Languages candidates may submit in
problemStatement	string	Full problem description (max 10,000 chars)
sampleInput	string	Visible sample input (max 2,000 chars)
sampleOutput	string	Expected output for sample input
solutionLanguage	string	Language of the reference solution
solutionCode	string	Reference implementation (max 20,000 chars)
solutionExplanation	string	Approach explanation (max 5,000 chars)
testCases	TestCase[]	Hidden test cases for automated validation

Coding problem JSON payload example:

```
{
  "problemId": "prob_001",
  "title": "Find the maximum element",
  "marks": 20,
  "allowedLanguages": ["Python", "JavaScript", "Java"],
  "problemStatement": "Given an array of integers, return the maximum value.",
  "sampleInput": "[3, 1, 4, 1, 5, 9, 2, 6]",
  "sampleOutput": "9",
  "solutionLanguage": "Python",
  "solutionCode": "def solve(arr): return max(arr)",
  "solutionExplanation": "Python's built-in max() runs in O(n).",
  "testCases": [
    { "input": "[1,2,3]", "expectedOutput": "3", "explanation": "Last element is max" }
  ]
}
```

2.7 Assessment Builder Flow

Custom assessments are built through a 4-step wizard (3 steps for NON-IT roles):

Step	Name	Description	NON-IT
1	Assessment Details	Job title, role type, marks, duration, difficulty, skills	Included
2	Knowledge Assessment	Add sections (single choice, T/F, short/long answer) and questions	Included
3	Skill Assessment	Add coding problems with test cases and reference solutions	Skipped
4	Review & Publish	Review full assessment; validate marks; publish to candidates	Included

03 Assigning Assessments

Assignments link published assessments to groups of candidates, called batches. The platform automatically deduplicates, generates unique session tokens, and notifies candidates.

3.1 Assignment Fields

Field	Type	Description
assignmentId	string	Unique assignment batch ID (assgn_xxxx)
assessmentId	string	The assessment being assigned
orgId	string	Owning organisation
batchName	string	Human label for this batch
tag	string	Optional grouping tag (e.g. frontend-hiring)
candidates	CandidateRecord[]	List of candidates in this batch
totalCandidates	number	Final count after deduplication
duplicatesRemoved	number	How many emails were skipped (already assigned)
alreadyAssigned	string[]	Emails that were in a previous batch for this assessment
createdAt	ISO 8601 date	Batch creation timestamp

3.2 Candidate Record

Field	Type	Description
candidateId	string	Unique candidate identifier
name	string	Full name
email	string	Email address (used as unique identifier per assessment)
status	string	"pending" "started" "completed" "expired"
assignedAt	ISO 8601 date	When the candidate was added to the batch

3.3 Two-Step Assignment Flow

Step 1 — Duplicate Check (Preview)

Before creating a batch, query which submitted emails are already assigned to the same assessment. This allows the UI to warn the user before committing.

```
POST /api/v1/assessments/{assessmentId}/assign/check
Authorization: Bearer {accessToken}
Content-Type: application/json

{
  "orgId": "RORG123456",
  "emails": ["alice@example.com", "bob@example.com", "carol@example.com"]
}

// Response:
{
  "success": true,
  "data": {
    "alreadyAssigned": ["bob@example.com"]
  }
}
```

Step 2 — Create Assignment Batch

Submit the confirmed candidate list. Duplicates are automatically removed server-side even if not pre-checked.

```
POST /api/v1/assessments/{assessmentId}/assign
Authorization: Bearer {accessToken}
Content-Type: application/json

{
  "orgId": "RORG123456",
```

```

"batchName": "Batch June 2026",
"tag": "frontend-hiring",
"candidates": [
  { "name": "Alice Smith", "email": "alice@example.com" },
  { "name": "Carol Johnson", "email": "carol@example.com" }
]
}

```

What happens after assignment

Each candidate receives a unique session token scoped to this assessment.

Candidates access the assessment via a link containing their session token.

The system records all events: start time, answers, proctoring violations, submission.

3.4 Listing Assignments

All Batches for a Specific Assessment

```

GET /api/v1/assessments/{assessmentId}/assignments?orgId={orgId}
Authorization: Bearer {accessToken}

```

All Batches Org-Wide (Mode Filtered)

```

GET /api/v1/assessments/assignments?orgId={orgId}&mode={mode}
Authorization: Bearer {accessToken}

// mode=test shows only test-mode assessment assignments
// mode=live shows only live-mode assessment assignments

```

3.5 Candidate Session Lifecycle

Once assigned, each candidate has a session that progresses through the following statuses:

Status	Trigger	Description
registered	Assignment created	Candidate added; session token issued; not yet started
verified	Identity check passed	Candidate completed identity verification (face / ID)
in_progress	Assessment started	Active — candidate is answering questions
submitted	Candidate submits	All answers submitted; scoring initiated
timed_out	Duration limit reached	Auto-submitted when the timer expires
disqualified	Proctoring violation	Critical violation detected; session locked

3.6 Overview Dashboard API

The overview endpoint aggregates assessment and session data into dashboard-ready metrics, all filtered by mode.

```
GET /api/v1/assessments/overview?orgId={orgId}&mode={mode}
```

```
Authorization: Bearer {accessToken}
```

Response structure:

```
{
  "stats": {
    "totalAssessments": 42,
    "assignedCandidates": 1250,
    "completedAttempts": 980,
    "averagePassRate": 71
  },
  "funnel": {
    "created": 42,
    "assigned": 1250,
    "started": 1100,
    "completed": 980,
    "passed": 696
  }
}
```

```
},
"modeStats": [
{
  "mode":      "AI Powered",
  "subtitle":  "JD based assessments",
  "created":   20,
  "assigned":  600,
  "completed": 480,
  "avgScore":  74,
  "passRate":  73
},
// ... Customized, Smart AI
]
}
```

Metric	Calculation
averagePassRate	passed sessions / completed sessions x 100
Funnel %	Each stage count / assigned count x 100
passRate (mode)	passed sessions for that mode / completed x 100
avgScore (mode)	Average candidate score across completed sessions